Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3640310.3674103

RESEARCH-ARTICLE

Mutation Testing of Java Bytecode: A Model-Driven Approach

CHRISTOPH M BOCKISCH, Philipps University of Marburg, Marburg, Hessen, Germany

DENIZ EREN, Philipps University of Marburg, Marburg, Hessen, Germany

SASCHA LEHMANN, Philipps University of Marburg, Marburg, Hessen, Germany

DANIEL NEUFELD, Philipps University of Marburg, Marburg, Hessen, Germany

GABRIELE TAENTZER, Philipps University of Marburg, Marburg, Hessen, Germany

Open Access Support provided by:

Philipps University of Marburg

PDF Download
3640310.3674103.pdf
08 February 2026
Total Citations: 0
Total Downloads: 494

Published: 22 September 2024

Citation in BibTeX format

MODELS '24: ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems
September 22 - 27, 2024
Linz, Austria

Conference Sponsors:
SIGSOFT

MODELS '24: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (September 2024)
https://doi.org/10.1145/3640310.3674103
ISBN: 9798400705045



Mutation Testing of Java Bytecode: A Model-Driven Approach

Christoph Bockisch
bockisch@uni-marburg.de
Philipps-Universität Marburg
Marburg, Germany

Deniz Eren
erend@students.uni-marburg.de
Philipps-Universität Marburg
Marburg, Germany

Sascha Lehmann
lehmanns@student.uni-marburg.de
Philipps-Universität Marburg
Marburg, Germany

Daniel Neufeld
neufeld.97@gmx.de
Philipps-Universität Marburg
Marburg, Germany

Gabriele Taentzer
taentzer@uni-marburg.de
Philipps-Universität Marburg
Marburg, Germany

ABSTRACT

Mutation testing is an approach to checking the robustness of test suites. The program code is slightly changed by mutations to inject bugs. A test suite is robust enough if it finds such bugs. Mutation testing tools typically integrate sets of mutation operators such as, for example, swapping arithmetic operators; modern tools typically work with compiled code such as Java bytecode. The mutations must be defined in such a way that the mutated program can still be loaded and executed. The results of mutation tests depend directly on the possible mutations. More advanced mutations and even domain-specific mutations can pose another challenge to the test suite. Since the classical, non-model-based mutation testing tools do not support the specification of advanced mutation operators well, we propose a model-driven approach where mutations of Java bytecode can be flexibly defined by model transformation. Our approach also provides advanced mutation operators for modifying object-oriented structures, Java-specific properties and API method calls, making it the only mutation testing tool for Java bytecode that supports such mutations. To further improve the effectiveness of mutation testing, mutants are generated only for bytecode that is covered by tests. Our approach is implemented in the MMT tool. It has been evaluated against non-model-based mutation testing tools for its ability to generate mutants close to real bugs. The experiments make use of Defects4J, a well-established collection of real-world Java projects with reproducible bugs.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**; **Object oriented languages**; **Domain specific languages**.

KEYWORDS

Mutation testing, Java bytecode, Model transformation



This work is licensed under a Creative Commons Attribution International 4.0 License.
MODELS '24, September 22–27, 2024, Linz, Austria
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0504-5/24/09
https://doi.org/10.1145/3640310.3674103

ACM Reference Format:
Christoph Bockisch, Deniz Eren, Sascha Lehmann, Daniel Neufeld, and Gabriele Taentzer. 2024. Mutation Testing of Java Bytecode: A Model-Driven Approach. In *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS '24), September 22–27, 2024, Linz,Austria.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3640310.3674103

1 INTRODUCTION

Mutation testing is a technique for checking the quality of a test suite. The assumption is that a program is well tested if its tests can eliminate most of the bugs injected into the program by mutations [24]. The mutations to be applied to the program are defined by mutation operators, which describe small changes to the program code that do not violate the constraints of the programming language. For example, a mutation operator can change the value, but not the type, of an expression. Since the effectiveness of a mutation testing approach depends directly on the potential of its mutation operators to find missing tests in the test suite, their variance and strength play a central role.

Mutations for Java programs can be performed either on the source code or on the bytecode. For both cases, there are tools that support writing code transformations, such as Polyglot [22] for source code, and ASM [9] or BCEL [5] for bytecode. While mutating source code is often straightforward, there are drawbacks to this approach: Each mutated program must be recompiled, which can lead to significant compilation overhead [15]. In addition, relevant information is often implicit in the source code; the absence of such information during mutation can easily lead to illegal code, and solving this problem may require copying the functionality of a compiler. An example is the invocation of overloaded methods, where the compiler performs a type analysis to select the correct implementation; in bytecode, the method reference is already unique. Therefore, newer mutation testing tools mainly use bytecode mutation.

Looking at Java bytecode mutation tools like Jumble [13] and PITest [11], they typically provide mutation operators for manipulating arithmetic operators such as replacing '+' with '-', relational operators such as replacing '==' with '!=', operators that replace return values, and operators that change the values of variables and constants. However, they do not provide mutation operators for manipulating the object-oriented structure, Java-specific properties of a program and library method calls, and therefore cannot assess the ability of a test suite to detect errors in the use of such concepts.

Example. Fig. 1 illustrates this technique with an example that is currently only supported by our tool among modern mutation testing tools for Java bytecode. The figure shows a code snippet from a Java project (top) and a corresponding test case (bottom). The test invokes the method getMediaInfo of ImageMessage, which in turn returns the result of prettyPrint. It is asserted that the method returns an actual string and not **null**.

One possible mutation is to remove the method definition of prettyPrint from the class ImageMessage (marked by the red box). The resulting code is still executable because a concrete implementation of the method prettyPrint is inherited from the superclass AbstractDataMessage. While the result "Message has no text." is obviously undesirable in the case of the mutant, the test is successful when run with both, the original version and the mutant.

```java
package message;

public abstract class AbstractDataMessage
        extends AbstractMessage
        implements DataMessage {

    protected AbstractDataMessage(
            User sender, User recipient) {
        super(sender, recipient);
    }

    public String prettyPrint() {
        return "Message has no text.";
    }
}
```

```java
package message;

public class ImageMessage
        extends AbstractDataMessage {

    private Image image;

    public ImageMessage(String mediaInfo,

    public String getMediaInfo() {
        return this.prettyPrint();
    }

    @Override
    public String prettyPrint() {
        return "Image: " + image.prettyPrint();
    }
}
```

a) Developer-written application code.

```java
package message;
import org.junit.Assert;


public class MessageTest {

    User sender = new User(10, "Daniel97", "Daniel", "Neufeld");
    User recipient = new User(11, "MrLel", "Peter", "Müller");

    @Test
    public void imageMessageTest() {
        ImageMessage testMessage = new ImageMessage("Picture", new Image(), sender, recipient);
        String mediaInfo = testMessage.getMediaInfo();
        Assert.assertNotNull(mediaInfo);
    }
}
```

b) Developer-written test case code.

Figure 1: Example code to which mutation testing can be applied. When the code in the red box is removed, the test still succeeds.

Fig. 2 shows the details of the mutation that are reported by our mutation testing tool MMT for the described mutant. It shows the type of mutation, where it was applied, and that it was not detected by any test case, indicating that the test suite is too weak. This conclusion is obviously correct, since the test does not actually inspect the result value. Writing a better assertion later, we discover that

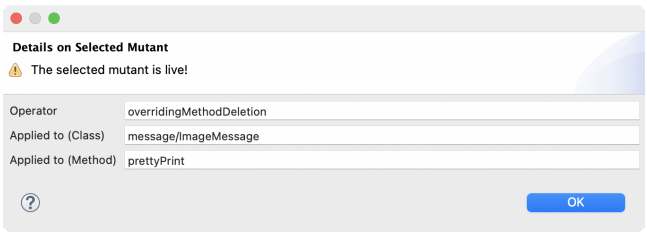


Figure 2: Details reported on a specific mutant, indicating a non-detected mutant.

the method prettyPrint in ImageMessage is probably wrong because it contains the inappropriate dummy message returned by the super implementation.

One reason we believe that current bytecode mutation testing tools do not provide advanced mutations is the complexity of bytecode consistency rules, and the fact that such advanced program constructs may be distributed across multiple locations in the bytecode; for example, whether a method is private or not affects all of its call locations. The bytecode manipulation tools mentioned above abstract from only some of these complexities. Consequently, developers of mutation operators must deal with a fixed, relatively low level of abstraction. If there were an easy-to-use approach for specifying mutation operators at different levels of bytecode abstraction, the benefits of bytecode-level mutation would outweigh the complexity, and extending bytecode mutation operators to advanced program constructs would be feasible. We believe that a model-driven approach can help here, and therefore propose our approach and tool called *Model-based Mutation Testing* (MMT).

To allow the developer to choose the right level of abstraction, a model-driven approach to transforming Java bytecode has been developed in [7]. It builds on the Mod-BEAM approach [30], where Java bytecode is represented as a model, along with a parser that reads in bytecode and creates a model, and a printer that creates bytecode from a model. In [7], the rules of the bytecode verifier (which is used by the Java Virtual Machine to identify inconsistencies in the bytecode comparable to compiler errors) are specified as OCL constraints to provide a basis for automatically checking the syntactic correctness of program transformations. This will help us to specify mutation operators that generate only syntactically correct programs. The contributions of this paper are as follows:

- We use the model-driven approach to program transformations in [7] to specify Java bytecode mutations. A large variety of mutation operators are specified as model transformation rules using Henshin [4], a model transformation engine based on the Eclipse Modeling Framework. Since the Mod-BEAM metamodel abstracts some technical details from the bytecode, mutation operators can be implemented with minimal knowledge of Java bytecode. The set of implemented operators includes method-level operators, such as replacing arithmetic operators, values, or method calls, and class-level operators, such as deleting overridden methods, type casts, or initializing objects with the parent type. The class-level operators in particular require the transformation of graph-like structures, which is not possible when using program transformation libraries such as ASM and BCEL.
- Our mutation testing tool MMT translates Java bytecode into a model, automatically applies the specified mutation operators to this bytecode model, translates the mutated model back into code, runs the test suite on the mutated code, and summarizes the errors caused by the mutation that were missed by the test suite. For ease of use, we provide MMT as an Eclipse plug-in that integrates with the graphical user interface of the Java Development Tools.
- Not all generated mutants are really useful for checking the robustness of a test suite in the sense that the suite is able to find real bugs. To reduce the amount of mutants

generated to the interesting ones, we have developed static and dynamic strategies that reduce the generation of mutants to the more interesting ones. These strategies are able to generate mutants only for code that is covered by tests, or for methods that are more complex.
- A comparison of MMT with non-model-based mutation testing tools shows that it is the only tool for Java bytecode that provides advanced mutation operators. This wider range of mutation operators can be seen as a necessary prerequisite for generating more diverse mutants. Similar to non-model-based tools, MMT proved to be usable for mutation analysis on real projects; nevertheless, presently, the non-model-based tools are usually faster.

The paper is organized as follows: In Section 2, we present our approach to mutation testing of Java bytecode, and in Section 3 we give an overview of the tool support in MMT. Section 4 presents the evaluation of our approach. Section 5 discusses the related work, and Section 6 draws a conclusion.

2 APPROACH

In this section, we present our model-driven approach to mutation testing of Java bytecode. First, we give an overview of the mutation testing workflow in general using a model-driven approach. Then, we briefly recall the Mod-BEAM approach, which provides a model for Java bytecode. Program mutation can thus be performed as model transformation. Mutation operators should be designed to always produce syntactically correct and executable bytecode. Finally, we show how the generation of mutants can be reduced in a meaningful way.

2.1 Workflow

The concept of mutation testing can be used to measure the quality of a test suite for a given program. This does not limit generality, since tests that fail can be disabled. A mutation testing tool repeatedly mutates the program by injecting an error into the code. The test suite is then run on each of these mutated programs. If there is a test that fails on the mutated program, that mutant is *detected* or *killed*. Mutations that are not detected are called *living*.

To mutate Java bytecode in our model-driven approach, it is first parsed and translated into a bytecode model. This model is then transformed by applying a model transformation rule or, if the mutation is more complex, by applying an entire transformation unit. The resulting model is then translated back to bytecode. We use the Mod-BEAM approach [7] based on the Eclipse Modeling Framework (EMF) [1] to translate between bytecode and model, which is presented below. In principle, any model transformation approach that works on EMF models can be used to specify and perform model transformations. We use Henshin [4], a model transformation language and environment for EMF, that is based on graph transformation concepts and supports formal validation.

Defining mutations as model transformations in conjunction with the characteristics of Mod-BEAM leverages the extensibility of our approach with new mutation operators for several reasons. Mutation operators can be written in a declarative way, which makes them very readable. Furthermore, all model elements can be accessed uniformly, making it easy to implement mutations that apply to multiple program elements, even if they are far apart in the code, such as manipulating the call to a method defined in another class. To maintain the consistency of the bytecode model, the application of mutations can be context-sensitive. Finally, transformations are defined in a separate formalism and dynamically loaded. Therefore, new mutation operators can be easily added without recompiling our tool.

2.2 A Metamodel for Java Bytecode

To facilitate the implementation of mutations as model transformations, we build on the Mod-BEAM approach [7, 30], which stands for "Modular Bytecode Engineering and Analysis with Models". This approach provides an Ecore metamodel capable of fully representing arbitrary Java bytecode. We briefly summarize the aspects of the metamodel that are most relevant to the implementation of mutations.

In the metamodel, Project is the root of the metamodel and contains Clazz[1] eObjects, each of which represents a `.class`-file. These in turn contain nested Method and Field eObjects. The code of a method is stored as a control-flow graph consisting of Instruction eObjects.

The abstract EClass Instruction is the root of a type hierarchy that contains a concrete EClass for each possible Java bytecode instruction. Each instruction type can have specific attributes; for example, an instruction to create a new object has the attribute TypeReference. All instructions have in common that they contain a reference to outgoing control-flow edges represented by a subtype of the EClass ControlFlowEdge. This EClass has two properties, namely a reference to the start instruction and a reference to the end instruction. Possible subtypes are UnconditionalEdge, ConditionalEdge and ExceptionalEdge.

In [7], the semantic rules of the bytecode verifier have been specified as OCL [23] constraints on top of the bytecode metamodel. Mod-BEAM is implemented as an Eclipse plug-in that can automatically generate instance models for all the compiled code in an Eclipse Java project. Likewise, the plug-in can automatically generate equivalent `.class` files from an instance model after it has been transformed.

2.3 Mutation Testing with Model Transformation

A mutation of Java bytecode is typically a small change to the bytecode. The assumption here is that mutations are applied to a correctly functioning program, and that any mutation will usually make the program incorrect. However, the mutation should not result in a syntax or type error. Such an error would be detected by the Java Virtual Machine when loading the class and would prevent the execution of the test suite altogether. Conversely, the changes introduced should be subtle and only be detected when a test case uses the correct test data and/or assertion. For these reasons, mutations typically represent code changes that software developers are likely to encounter. For example, it is easy to confuse "<" with "<=" in a termination condition. A typical example where class-level mutation can be useful is as follows:

[1]This spelling is used avoid confusion with the type java.lang.Class.

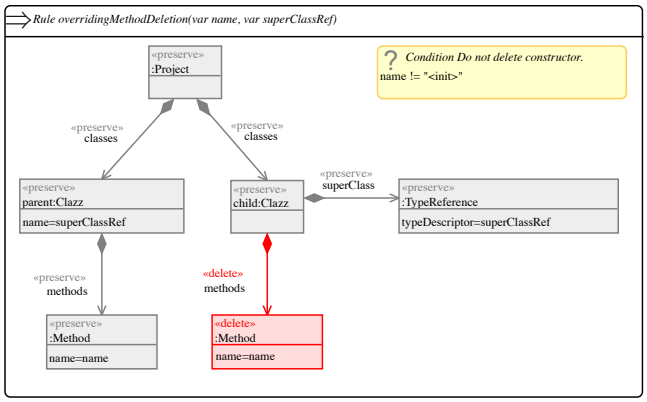


Figure 3: Mutation operator for deleting an overriding method specified as Henshin transformation rule.

Example. Developers can easily forget to override a method of a superclass, which can be simulated by a mutation operator by removing the method in the subclass. To satisfy our goals that mutated programs are still syntactically correct and contain no type errors, the mutation operator must express a condition that involves information from two different classes: A method definition can be safely removed from the *child class* if a method of the same signature is defined in the *parent class*—in this case, a call to the method on an object of the child can still be executed.

Fig. 3 shows a Henshin rule specifying the deletion of an overriding method in Java bytecode. It specifies a mutation operator that is applicable in terms of a pattern, represented by the gray model objects with the «preserve» stereotype, that must be present in the model: There are two relevant classes contained in a common project; the class referenced by `child` has a superclass whose name is referenced by the variable `superClassRef`. The name of the second class is the same as the value of the variable `superClassRef`. In addition, both classes contain a method named `name`.[2] The red object node with the stereotype «delete» means that a method named `name` in the subclass is to be removed from the model. All adjacent references will also be deleted. The application condition (in yellow) checks that the method name is not `<init>`, which is reserved for constructors in bytecode. The mutation operator also has a parameter list that contains the name of the overridden method and the name of the superclass. In the case of an arbitrary mutation, these parameters are automatically set by the rule match.

The effect of a concrete application of this mutation operator is illustrated in Fig. 4, which shows a snippet from the Mod-BEAM model of the class in Fig. 1, in an abstract graphical syntax. The model contains only objects that are directly involved in the mutation that applies the operator in Fig. 3, plus a few other directly connected objects to show a bit more context. The objects shown as gray boxes are those that are expected to be present (corresponding to the «preserve» stereotype in the Henshin rule). The bold blue arrows indicate which attribute values of related objects must match for the mutation operator to apply. The `Method` object shown in red

[2]To support overloading, we could also require matching parameter lists, which we omit here for brevity.

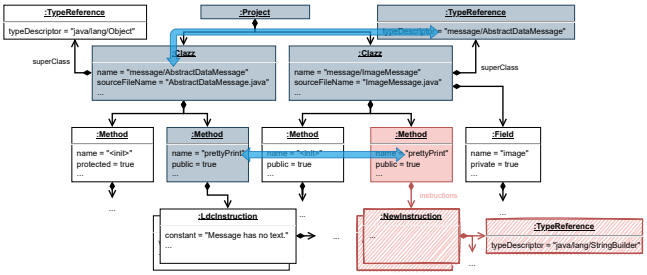


Figure 4: A snippet from the instance model in abstract graphical syntax corresponding to the source code in Fig. 1, showing the match of the mutation operator in Fig. 3.

is deleted. The instructions which it contains are red and shaded because they are not explicitly mentioned in the transformation rule but they are detached and become garbage, when their container is deleted.

2.4 Implemented Mutation Operators

MMT provides a wide range of mutation operators. A classic set of mutation operators are arithmetic and relational operator replacements, such as replacing "+" with "-" and "==" with "!=". These operators are typically found in mutation testing approaches. Operators that replace program parts such as conditionals, simple statements, and method calls, with simple values are also common in mutation testing and they are also provided by MMT. In contrast, class-level mutation operators, as presented in [25], are not common for mutation testing tools, at least not for bytecode mutation. This set includes operators that deal with inheritance, polymorphism, and Java-specific properties. We have also implemented mutation operators for modifying method calls of the Java Collection Framework, in particular, and of any API, in general. Note that most of these operators have an inverse, which we have implemented as well.

The more advanced mutation operators are usually defined from a source-code perspective, since mutations should represent errors that developers are likely to make, such as the declaration of a local variable that accidentally overshadows a field declaration. What is a simple line of code inserted by a source-code mutation (the variable declaration in the example) may require the mutation of multiple locations in the bytecode (such as replacing all affected field accesses with accesses to the local variable). This is because the source code is at a more abstract level than the bytecode, and many abstractions are resolved during compilation. This may also be a reason why some of the advanced mutation operators discussed above are not found in modern bytecode-based mutation testing tools. We were still able to implement them in MMT because Mod-BEAM provides a model at a higher level of abstraction than the bytecode itself. In summary, the list of operators in MMT is as follows:

(1) **Data types:** change of arithmetic and relational operators
(2) **Replacements:** replacement of values, conditionals, simple statements, or method calls with default values (or nothing for void methods), parameters; swapping of parameters/method calls, removement of increment

(3) **Java-specific:** insertion/deletion of `this`, `static` or member variable initialization; exchange of access methods, modifier methods, reference or contents comparison
(4) **Object orientation:** Showing/hiding of subclass variable; deletion/insertion of super or overriding method; change of position of overriding method, initialization of object with parent/child type, declaration of member variable/parameter with parent/child type, change of cast type to parent/child, deletion of type cast, exchange of body of overloaded method; change of arguments in overloaded methods
(5) **For any API:** replacement of parameter/method call; swapping of parameters/method calls

In total, we have implemented 89 mutation operators in MMT. When mutating Java bytecode, we want to make sure that the mutated code is still syntactically correct and executable. Mutation rules can be statically checked for preserving the EMF constraints [6]. If they do so, the resulting mutant is a valid EMF model that can be viewed and edited in model editors. We checked all 89 implemented mutation operators for this property and found that they all preserve the EMF constraints. Since the bytecode verifier rules were formalized as OCL constraints in [7], any model transformation can be accompanied by a post-processing step that checks the corresponding OCL constraints. Alternatively, a model transformation rule can also be extended to include application conditions. They control the applicability of the transformation rule so that all possible applications lead to transformation results that satisfy the given set of constraints. The construction of such application conditions from a given set of OCL constraints is implemented in an Eclipse-based tool called OCL2AC [21].

2.5 Reduction Strategies for Mutant Generation

Not all generated mutants are really useful for checking the robustness of a test suite. For example, generating mutants for source-code fragments that are not tested at all is useless, because such mutants would only provide the same information as test coverage analysis, which is much simpler. On the other hand, since complex methods tend to have a higher potential for containing bugs, mutation testing of such methods is of high interest.

To deal with the variable importance of mutants, we have developed *static and dynamic reduction strategies for mutant generation*. That is, the code is analyzed either statically or dynamically, and the analysis result is attached to the bytecode model elements as annotation objects. For this purpose, an extension of the Mod-Beam metamodel is defined, which contains a single class called `Annotation`. This class defines a reference to the class `ModBeamObject`, which is a superclass of all classes in the Mod-Beam metamodel. This allows annotations to be attached to all model elements. The `Annotation` class has the two attributes `key` and `value`, which store the name and result of the analysis that was used. A Mod-Beam object can have multiple annotations added by different analyses in this way.

In the *static strategy*, the bytecode model is inspected to compute the analysis results. An example is the calculation of the cyclomatic complexity of a method. In this case, each `Method` object is given an `Annotation` with the key `"complexity"` and the value corresponding to the cyclomatic complexity of the method. The annotated model is then used for mutation testing in the usual way, with the only difference that the mutation operators can also use the annotations, e.g. to check whether the cyclomatic complexity is greater than four, as shown in the Figure 5. The dashed blue box marks the required addition for this restriction. To avoid cluttering the mutation operators with such boilerplate definitions, the MMT framework allows the analysis implementations to automatically extend the mutation operators. Thus, the definitions in the dashed boxes do not actually need to be written by the developer of a mutation operator.

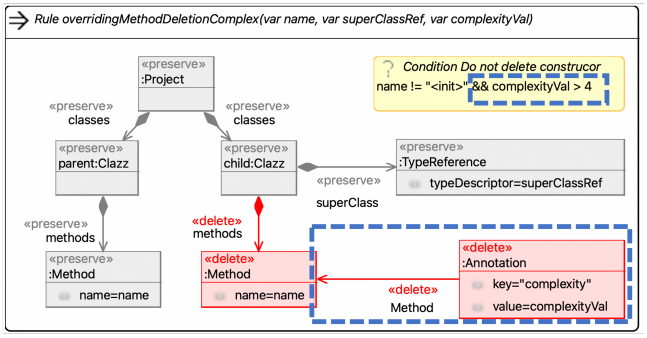


Figure 5: A mutation operator that is only applied to complex methods.

In the *dynamic strategy*, the bytecode model is first instrumented with additional code that is needed to perform additional computations, such as tracking methods executed by a given test suite. The result of such a dynamic analysis is then also stored in the model annotations. Instrumentation is performed in two steps: First, a model transformation is applied to the original bytecode model to instrument the code. For example, to collect code coverage information, calls to a log method are inserted at the beginning of each original method. Second, the instrumented bytecode is generated, and then the test suite is executed on the instrumented code.

As a mutation testing tool, MMT already has the ability to execute the application's test suite on a transformed version of the application code. During this initial test execution, data is dynamically collected, such as a list of methods that are executed during the test run.

The collected information is then used to attach annotations to the bytecode model. This step is in principle the same as in the static strategy, the only difference being the way in which the information to be included in the annotation is determined; the annotations are inserted into the original version of the bytecode model (i.e., the instrumented bytecode is discarded after the data collection) and can, thus, be used by the mutation operator. The latter is extended in the same way as discussed for the static approach to reducing mutation generation. However, in the dynamic coverage analysis example, the key of the annotation would be `"covered"` and the value would be boolean. Thus, the application condition would restrict mutation operators to model elements with a coverage annotation and a value of `"true"` to mutate code only within covered methods.

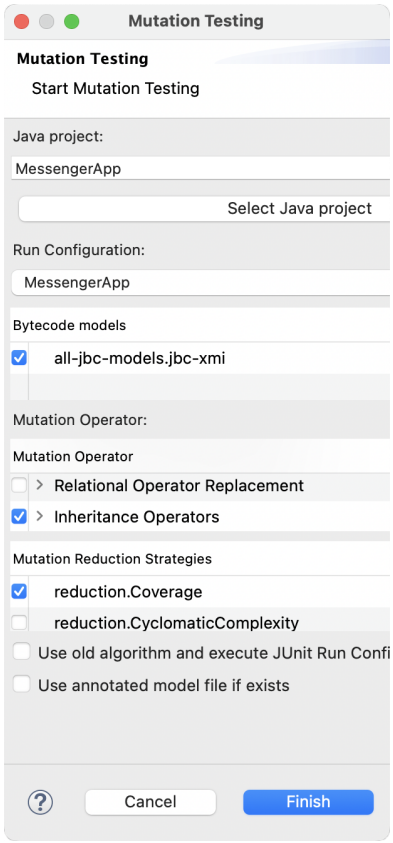


Figure 6: The Mutation Testing dialog.

3 TOOL SUPPORT

The Model-based Mutation Testing (MMT) tool[3] is an Eclipse plug-in. It supports the mutation testing of Java bytecode using a predefined set of mutation operators. This set of operators can be managed and extended by specifying custom mutation operators for Java bytecode. A demonstration of the main features of MMT is given in [8].

To perform the mutation analysis, a user must configure the following in the Mutation Testing dialog box (cf. Fig. 6): (1) a Java project from the Eclipse workspace, which should include JUnit tests, (2) a run configuration to run the test suite, (3) a selection of which of MMT's mutation operators to use, and (4) optionally, a selection of reduction strategies to apply.

MMT comes with 89 mutation operators in nine categories for users to choose from. In addition, users can add custom mutation operators by simply specifying the appropriate operators with Henshin and adding the corresponding transformation files to the MMT plugin preferences. Such configured custom mutation operators can also be selected in the Mutation Testing dialog.

After the mutation testing analysis has been configured and executed, the result is displayed in the interactive Mutation Testing View, see Fig. 7. This view contains one line for each mutant, along with information about which mutation operator was applied (1) and where in the bytecode (2). For mutants that were detected by the test suite, it also shows the test that killed it (3). Mutants that were not detected are marked with "Not detected!" (4).

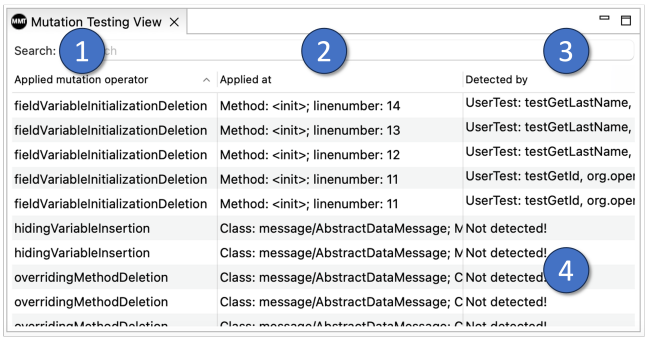


Figure 7: The Mutation Testing view.

4 EVALUATION

We will present two evaluations: First, we evaluate how effective the reduction strategies are for mutant generation. Second, we compare MMT with other mutation testing tools in terms of their ability to find real bugs. The evaluation data for the first experiment[4] and for the second experiment[5] are available online.

All of our evaluations will use selected Java projects from the Defects4J benchmark [16]. Defects4J is a database of Java projects that provides real bugs to enable reproducible studies in software testing research. For each bug of a project P, the database has a project version Pb where the bug still exists and causes a test to fail, and a version Pf where it is fixed and all tests pass. We always use the fixed versions of these projects for our experiments, mutating them to simulate bugs. For our experiments, we selected three projects, namely Cli, Codec, and JSoup, and a subset of project versions for each of them.

We chose these three projects from Defects4J because the patches are diverse enough to apply many different mutation operators, especially the class-level operators, and the projects are compatible with all three mutation testing tools we considered. The key metrics of some selected versions of these projects, used in multiple experiments, are listed in the Table 1 to give an idea of the size of these projects. In particular, the last two columns show the number of elements in the corresponding Mod-BEAM models, the total number of elements (eObjects), and the number of elements in the patched classes and their super- and subclasses.

4.1 Effectiveness of Reduction Strategies for Mutant Generation

Our first experiment examines the effect of the reduction strategies implemented for mutant generation in MMT: *How do the reduction*

[3]The MMT tool has passed the Artifact Evaluation of Models'24 and received the badges *Available* and *Evaluated* at the level *Reusable*. The reviewed version can be retrieved from https://doi.org/10.5281/zenodo.13138451. The most recent version can be retrieved from our Git repository at https://gitlab.uni-marburg.de/fb12/plt/modbeam-mt/mmt.

[4]https://gitlab.uni-marburg.de/fb12/plt/modbeam-mt/benchmarks/mutantreductionstrategyevaluationdata

[5]https://gitlab.uni-marburg.de/fb12/plt/modbeam-mt/benchmarks/mutationtestingtoolcomparison

Table 1: Selected Defects4J projects and versions used in the evaluation: Size of source code in KLOC, number of tests, size of test code in KLOC, total number of eObjects, number of eObjects in patched classes + super/subclasses

Project	KLOC	#Tests	Tests KLOC	# eObject	# patched
Cli 33	14	361	5	63052	3744
Cli 38	15	371	5	20121	3329
Codec 1	13	287	5	101888	3868
Codec 14	38	622	14	85699	10666
JSoup 3	10	146	6	30672	13159
JSoup 8	18	227	9	121032	8536

strategies affect the effectiveness of the mutant generation in MMT? We randomly selected several versions of the Defects4J projects Cli and JSoup. For each version, we generated a Mod-BEAM model for the classes that contained a bug, along with their subclasses and their superclasses. With this strategy, we have identified the project classes that are relevant to the detected bug, and we can ensure that each of the mutation operators can be applied. (Note that some of the class-level operators also require subclasses or superclasses for their application.)

We let MMT generate mutants, first without any reduction strategy, then with the restriction that mutants are generated only for code that is covered by tests, and finally with the additional restriction that mutants are generated only for methods with a cyclomatic complexity of greater than 1, i.e., methods with at least one conditional. For each set of generated mutants, we filtered out the mutants that are close to a real bug. A mutant is said to be *close to a real bug* if it is killed by the same test case that discovered the bug and mutates a method that has been patched.

Because mutations of loop conditions can lead to infinite loops, MMT forcefully terminates test runs after a timeout of 10 seconds. If the test execution on a mutant takes longer than this, it will be aborted and considered as killed. As will be discussed at the end of of the following subsection, a timeout of 10 seconds was comfortably sufficient for all tests to finish normally, if no infinite loops were accidentally caused by mutations.

Table 2 shows the number of all mutants generated by MMT without and with applying the reduction strategies. To set these numbers in relation to important mutants, i.e. those that are close to a real bug, those numbers are also shown in Table 2. Note that this experiment was performed with an earlier version of MMT than the following experiments.

The results show that the number of mutants that are close to a real bug remains nearly the same after applying both reduction strategies. For code coverage, this means that most of these mutants are located in covered code, and mutant set reduction is effective because it does not remove these mutants. In JSoup 3 and JSoup 8, however, the mutants modify methods that are not fully covered. When calculating cyclomatic complexity, it is not unlikely that the number of mutants close to a real bug may decrease. This is the case if the real bug occurred in a method with a linear control flow. In summary, the reduction strategies seem to remove many of the less effective mutants without significantly affecting the set of mutants that are close to real bugs.

Table 2: Number of all mutants without and with reduction strategies (important mutants in brackets): Project with version number, number of all important mutants without reduction, with test-coverage reduction strategy, and with combined reduction strategy based on test coverage and complexity.

Project	# mutants	# cov. mutants	# cov. mutants in compl. methods
Cli 31	146(6)	121 (6)	89 (6)
Cli 38	95 (6)	90 (6)	85 (6)
JSoup 1	161 (5)	125 (5)	84 (5)
JSoup 3	318 (38)	257 (37)	171 (32)
JSoup 7	191 (9)	157 (9)	102 (7)
JSoup 8	228 (4)	176 (0)	104 (0)

4.2 Comparing MMT with Other Tools

We will compare MMT with Jumble and PIT, two mutation testing tools for Java bytecode that are not model-based. We have chosen Jumble and PIT because Jumble is one of the first mutation testing tools to work on Java bytecode and PIT is currently one of the most effective and efficient tools. [18] In addition, we compare MMT to Wodel-Test [12], since it also implements a model-driven approach to mutation testing. Unlike MMT, Wodel-Test mutates Java source code and it uses a new domain-specific language for specifying mutation operators.

Our central research question is as follows: *How do the considered tools perform in terms of detecting real bugs?* To answer this question we consider the mutants that can be generated by the tools. Does the set of generated mutants contain mutants that are close to the real bug, in the sense that they are killed by the bug-detecting test case, or contain mutated program code that overlaps with the corresponding patch? We will refer to these mutants as *important*.

The ability to generate important mutants depends very much on the set of mutation operators used and where the operators are applied. Therefore, we start our tool comparison by comparing the sets of mutation operators already available in the tools and ask: *How comprehensive is the set of available mutation operators? How extensible is the set of mutation operators?* Table 3 gives an overview of the mutation operator groups defined at the level of methods and classes.

Jumble uses BCEL to specify mutation operators, while PIT uses ASM. The ASM and BCEL libraries support the programmatic manipulation of Java bytecode, where a tree-like representation of the bytecode is traversed to find contextual information and then manipulated. Because program structures are generally graph-like, the specification of class-level mutation operators can become difficult with ASM and BCEL. This is probably why class-level operators are not provided in Jumble and PIT.

Wodel-Test comes along with a domain-specific language for specifying mutation operators. This language has been used to specify a variety of method-level operators. It remains open to what extent the Wodel specification language can also be used to specify class-level operators. MMT can reuse a model transformation language such as Henshin to specify new mutation operators.

While Henshin is currently integrated as a specification tool for mutation operators, the integration of other model transformation engines is also possible, as MMT uses them via an interface.

In summary, we can conclude that MMT is the only tool considered that provides class-level operators. However, compared to the other tools, some method-level operators are still missing. Mutation testing is often based on the hypotheses of the competent programmer, who does not make mistakes that lead to obvious bugs, but rather small inaccuracies that are not noticeable. In addition, there can be a coupling effect where the combination of several small mistakes can lead to a larger mistake. The mutation tests inject exactly these small mistakes. MMT's set of mutation operators is by no means complete. At the current stage of development, we were more interested in being able to specify very different kinds of mutation operators, especially at the class level. Therefore, we have implemented representational operators for most of the mutation operator groups listed on the left in Table 3. For example, the implementation strategy for logical operators would be the same as for arithmetic operators, namely replacing a single instruction in the bytecode. We have chosen representative operators with a focus on demonstrating the feasibility of different required specification strategies. We expect that the missing operators can be easily added in the near future.

Table 3: Comparison of mutation operators: "x" means that this type of operator is well supported; "no" means it is not supported.

Operators	MMT	Jumble	PIT	Wodel-Test
Method-level operators				
arithmetic repl. (+,-,*,%)	partly	x	x	x
conditional repl. (&&,||)	no	x	x	x
logical repl. (&,|,!)	no	x	x	x
relational repl. (==,!=,<,>)	partly	x	x	x
shift repl. («,»)	no	x	x	x
value replacement	x	x	x	partly
call replacement	partly	no	x	partly
increment repl. (i++,i- -)	x	x	x	x
return replacement	x	x	x	x
parameter exchange	x	no	x	no
Class-level operators				
Java-specific	x	no	no	no
polymorphism	x	no	no	no
inheritance	x	no	no	no
Operator specification				
specification language	MTL	BCEL	ASM	Wodel

Next, we examine how the mutation operators can be applied to bytecode in the tools under consideration. While MMT and Wodel-Test allow the individual selection of operators to be applied, PIT and Jumble support the selection of operator groups. To reduce the set of generated mutants to the interesting ones, MMT and PIT allow to select a subset of project classes for mutation testing, Jumble allows only the selection of one class. MMT also offers to reduce the mutant generation to covered code and in addition to methods with a cyclomatic complexity greater than 1. PIT automatically focuses mutant generation on covered code.

The reports on mutation tests also vary considerably between the different tools. All tools report on the number of mutants generated, the number of mutants detected, the number of mutants not detected, and the execution time. In addition, for each mutant, MMT and PIT provide the mutation operator applied and the location in the code where the operator is applied. Jumble provides this information as a csv output. For detected mutants, MMT also provides the names of the tests that detected the mutant.

To explore whether the tools under consideration are able to generate important mutants, we ran them on selected versions of the Defects4J projects Cli, Codec, and JSoup, as shown in Table 4. We tried hard to perform the experiments with Wodel-Test, but we were not able to test any of the selected projects Defects4J Cli, Codec or JSoup with this tool. Therefore, we do not consider Wodel-Test in the dynamic part of the comparison.

We ran the experiments with all mutation operators available in the respective tools. For fixes that affect more than one class, we ran the experiment once for each patched class, so some project versions are listed multiple times in the table. The patched classes under consideration are listed in the second column of Table 4. In MMT, we generated a Mod-BEAM model for a patched class along with its subclasses and superclasses. Although we could have included all patched classes and their hierarchies in one Mod-BEAM model, we chose to limit the mutations to a single class (and its inheritance hierarchy) in order to obtain results that are comparable to those of the other tools. For MMT, we decided to reduce the set of mutants to the methods covered by the test suite.

For each selected project version, we report in Table 4 whether MMT found an important mutant that is detected by the bug-detecting test case. Unlike MMT, PIT and Jumble do not report this information in their result views. So we only check this information for MMT. For all three mutation testing tools, we check whether they have generated mutants with mutated code that overlaps with the patched code to analyze whether there are generated mutants that are close enough to the reported defect.

Analyzing the test reports of MMT, Jumble and PIT, we see that Jumble generates mostly low-level mutants, at the level of methods using arithmetic, relational and conditional operations as well as changes to return values. In contrast, PIT and MMT generate more high-level mutants. Both tools generate mutants with different method or constructor calls. MMT also uses class-level mutation operators.

Further analysis of the test reports of MMT, Jumble and PIT shows that the mutation operators for generating the important mutants are also very different. While MMT mainly uses relational operators and call replacements, Jumble uses value and conditional replacements and PIT uses conditional replacements, value replacements, assignment replacements, and call replacements. There are a few cases, where MMT used a class-level operators. The operators `superKeywordInsertionM` and `superKeywordInsertionVar` were used for JSoup 3 - Element and Cli 22 - PosixParser.

In 21 out of 24 cases, MMT is effective in the sense that it is able to generate a mutant that is killed by the bug-detecting test case. All three tools are similarly effective at generating mutants that overlap with the patched code, with PIT being most effective in 22 out of 24 cases. In all cases, where no mutant with patch overlap is generated, the patches are small. The definition of further mutation operators

Table 4: Comparison of mutation testing tools: For which bugs are mutants generated so that they are killed by the bug-detecting test case, or contain mutated program code that overlaps with the corresponding patch?

Project	Patched class	MMT		Jumble	PIT
		killed?	patch?	patch?	patch?
Cli 22	PosixParser	✓	✓	✓	✓
Cli 27	OptionGroup	✓	✓	✓	✓
Cli 33	HelpFormatter	✓	✓	✓	✓
Cli 38	DefaultParser	✓	✓	✓	✓
Codec 1	Caverphone	✓	✓	-	✓
Codec 1	Metaphone	✓	✓	-	✓
Codec 1	SoundexUtils	✓	✓	✓	✓
Codec 4	Base64	✓	-	✓	✓
Codec 7	Base64	✓	-	✓	✓
Codec 8	Base64	-	-	-	-
Codec 9	Base64	✓	✓	✓	✓
Codec 10	Caverphone	✓	✓	✓	✓
Codec 13	DoubleMetaphone	✓	✓	✓	✓
Codec 13	StringUtils	✓	✓	✓	✓
Codec 14	Lang	✓	-	✓	-
Codec 14	PhoneticEngine	✓	✓	✓	✓
JSoup 3	Element	✓	✓	-	✓
JSoup 3	Parser	✓	✓	✓	✓
JSoup 4	Entities	✓	✓	✓	✓
JSoup 6	Entities	-	✓	✓	✓
JSoup 8	Node	✓	✓	✓	✓
JSoup 22	Element	✓	✓	✓	✓
JSoup 22	Elements	-	-	-	✓
JSoup 22	Node	✓	✓	✓	✓
Total		21	19	19	22

could also cover these cases. For example, a new constructor is created in the patch of JSoup 22 - Elements (Es). Jumble and MMT do not generate mutants that overlap with this patch, probably because they do not have mutation operators for manipulating constructor calls.

For two patched classes of Codec 1, all mutants generated by Jumble only mutate unpatched code. In contrast, MMT and PIT are able to generate mutants with code that overlaps with the patch in all cases. For Codec 14, MMT was able to generate mutants that were detected by the bug-detecting test case, but all failed to generate mutants with code that overlaps with patched code, similar to PIT. It appears that a test class is broken. However, Jumble was able to generate mutants that overlap with patched code.

Next, we examine the efficiency of the tools MMT, Jumble, and PIT. For the same selected versions of the Cli, Codec, and JSoup projects as above, we measured how many mutants were generated for the patched classes (and super- and subclasses in the case of MMT). We also measured the execution time of the corresponding mutation analysis in milliseconds. The results are shown in Table 5.

On average, MMT needs 4569 ms per mutant, Jumble needs 573 ms, and PIT only needs 56 ms. These execution times show that mutant generation in MMT, which includes the model transformation, translation of the transformed bytecode model to bytecode and execution of the test suite, takes two orders of magnitude more time

Table 5: Number of generated mutants and execution time in milliseconds in MMT, Jumble and PIT

Project	MMT		Jumble		PIT	
	# No.	Time	# No.	Time	# No.	Time
Cli 22	101	40034	42	11979	153	7000
Cli 27	14	6906	18	5123	68	1000
Cli 33	130	290064	147	60711	545	28000
Cli 38	140	106501	183	50373	557	11000
Codec 1, C	67	30197	109	29457	209	3000
Codec 1, M	96	79256	247	70187	564	16000
Codec 1, S	13	8717	25	8050	65	1000
Codec 4,B	76	611845	907	236065	726	23000
Codec 7,B	86	650316	910	236137	735	21000
Codec 8,B	76	576384	902	235522	707	18000
Codec 9,B	77	623048	902	232348	714	17000
Codec 10,C	69	51781	109	30143	209	4000
Codec 13, D	325	1259717	1380	445651	90	1000
Codec 13, C	4	60638	17	12645	48	11000
Codec 13, S	10	126556	39	11032	90	1000
Codec 14, L	18	77603	43	13004	108	74000
Codec 14, P	47	955325	123	50974	258	25000
JSoup 3,E	202	213649	177	63947	561	12000
JSoup 3,P	75	559544	117	43379	370	88000
JSoup 4,E	20	626982	12769	8243776	108	6000
JSoup 6,E	20	639836	12769	8531240	112	6000
JSoup 8,N	56	450488	84	33414	232	32000
JSoup 22,E	131	265695	212	82083	663	16000
JSoup 22,Es	49	73162	116	43545	248	5000
JSoup 22,N	98	753940	138	58225	414	55000
Average	80	365527	1299	744140	342	19280

than a single pass in PIT. One explanation for the high execution times of MMT is the fact that, unlike the other tools, MMT considers super- and sub-classes for mutation. This is true for Cli 22, JSoup 3 - Element, JSoup 8 - Node, JSoup22 - Element and JSoup 22 - Node. Another source of high execution times is that the bytecode for the whole model is generated after each generation step, although only a small part of the model is changed.

Finally, the way in which MMT currently handles timeouts also contributes to the slow average execution time: A timeout is necessary, because mutating the termination condition of loops may turn them into infinite loops. To detect infinite loops, MMT currently simply interrupts tests simply after 10 seconds. The time per mutation as measured for PIT is an upper bound on the raw execution time of the test suites. Looking at the above figures, we can see that a timeout 10 seconds is actually three orders of magnitude more than the test suite takes to complete if no infinite loops are accidentally introduced. If mutations cause many infinite loops, the average execution time is therefore significantly slowed down by our pessimistic choice of the timeout.

For these reasons, we are planning several performance optimizations in the future. Most importantly, we, first, plan to implement incremental bytecode generation after mutant generation. Since mutations are typically localized to one or a few classes, not having to regenerate bytecode for unchanged classes will significantly speed up the mutation process. Second, we will adopt an adaptive

strategy for detecting infinite loops by first measuring the execution time of the unmodified test suite and using that time plus a few milliseconds as a buffer for the timeout.

4.3 Threats to Validity

We attempted to mitigate threats with respect to external validity by using Defects4J, a publicly available benchmark developed independently of our work and consisting of several real projects with real bugs. We ran our experiments on the fixed (clean) program versions of Defects4J instead of the buggy versions, which may pose a threat to internal validity [10], because the existing Java tools only work on passing test suites. To mitigate this threat, we focused our comparison of mutation testing tools on generating important mutants, i.e., mutants that are killed by the bug detecting test cases or contain mutated code that overlaps with the corresponding patches. Another threat to internal validity may be the manual checking for overlap between mutated code and patches. We have double-checked the results of this manual analysis.

4.4 Lessons Learned

The use of MMT, Wodel-Test, Jumble, and PIT for mutation analysis in selected Defects4J projects has led to the following lessons learned:

Lesson 1: *The set of mutants can be effectively reduced to those being covered by tests without significantly affecting the set of mutants close to real bugs.* Our results indicate that code coverage of mutated methods is essential for mutants being close to real bugs assuming that the bugs are all in covered code. This relationship was also found for the mutation testing tool PIT [29].

Lesson 2: *Model-based approaches support the transformation of graph-like structures rather than abstract syntax trees, allowing the specification of complex mutation operators.* We can experiment with new operators because they can be specified flexibly. In the future, we plan to analyze the research on programming errors such as API misuse [2, 3] to explore new mutation operators. The open question is whether known programming errors can help to specify operators that generate mutants that are close to real bugs.

Lesson 3: *Model-based approaches require translation between model and code, which takes time. Therefore, non-model-based approaches are usually faster.* Model-based approaches, such as Wodel-Test and MMT, use program models (i.e., source code models or bytecode models) to generate mutants. While they may contain more information that can be used advantageously for mutant generation, they, or at least their changes, must be translated back into code. In the future, we will implement bytecode generation incrementally after mutant generation so that the execution times of MMT are expected to decrease significantly.

In conclusion, model-based approaches to mutation testing appear to be promising prototyping tools for the development of advanced mutation operators. Once operators are identified as useful, they should also be implemented in non-model-based approaches to be available for efficient use in real projects.

5 RELATED WORK

We discuss related work in three directions: First we consider other mutation testing tools for Java, in particular for Java bytecode, then we compare with other model-based approaches to mutation testing, and finally, we consider other comparisons of mutation testing tools.

5.1 Mutation Testing Tools for Java

Javalanche [28], Jumble [14], LittleDarwin [27], Major [15], PIT [11], and Mujava [25] are several existing approaches and tools for mutation testing of Java programs. Since they are mainly designed for fast mutation testing, Javalanche, Major, and PIT work directly on Java bytecode, only Mujava was developed for Java source code. All bytecode mutation testing tools work only with simple mutation operators, for which it is easy to guarantee executable programs as output, and do not provide special facilities to extend the set of mutation operators such as a domain-specific language. Only Mujava and our approach MMT provide mutation operators on the level of classes using inheritance concepts and Java-specific features. MMT is the only tool that supports complex operators for bytecode, which is possible because mutations are transformations of graph-like structures and not of trees.

5.2 Model-Driven Approaches to Mutation Testing

There are two related MDE-based approaches to mutation testing presented in [20] and [12]. While Mottu et al. [20] present a mutation testing approach for model transformation programs only, Gomey-Abajo et al. [12] present a language-independent approach to mutation testing called Wodel. Wodel-Test is a model-based framework for mutation testing implemented as an Eclipse plug-in like MMT. It includes a specification language for mutation operators that has been used, among other specifications, to specify 77 method-level mutation operators for Java source code (not bytecode) such as arithmetic and relational operators. Wodel appears to be well suited for specifying these mutation operators. To ensure that the specified operators satisfy some form of correctness, Wodel (as MMT) supports post-processing of mutated models. Static checks on mutation operators, such as checking for EMF constraints, are mentioned but not explicitly reported in the experiments in [12].

5.3 Comparison of Mutation Testing Tools

Now we relate our work to other comparisons of mutation testing tools for Java bytecode. [19] compares PIT with Major and Mujava along the number of mutants and the mutation score. This paper focuses on the set of mutation operators used. The authors found that an extended set of operators has more potential to generate mutants of interest. However, generating more mutants also often means generating more equivalent or redundant mutants. Mutants are equivalent if they are syntactically different from the original program, but semantically equivalent so that the test suite behaves the same way on them. A mutation testing tool should generate as few equivalent mutants as possible. To solve the equivalent mutant problem, several mutant selection strategies have been proposed, such as mutant sampling and selective mutation. Another problem with mutation testing is the generation of redundant mutants. Probably the only technique capable of identifying such mutants

is Trivial Compiler Equivalence [26], which is based on compiler optimizations.

Kintis et al. [18] compare PIT, Mujava, Major, and a research version of PIT on a selected set of Defects4J projects. Instead of just comparing the number of mutants generated, the number of mutants detected, and the mutation score, the tools are compared along the real bugs as they occur in the Defects4J projects. A real bug is coupled to a mutant if tests that detect the mutant will also detect the bug, i.e., if tests that fail against the mutant will also fail against the buggy system. In [17], Just et al. presented a first study showing the relationship between mutants and real bugs. The relationship to real bugs is a major step forward in comparing mutation testing tools, since generating more mutants is not automatically better, but the right mutants have to be generated. Our comparison of mutation testing tools also takes this relationship into account, since we focus on generating important mutants, i.e., mutants that are closely related to real bugs. In [18], Just el al. define a bug to be revealed if there is at least one mutant that is only detected by the triggering test cases for that particular bug. To make the verification of the criterion easier, we have redefined it as follows: A bug is revealed if there is at least one mutant that is detected by the triggering test cases for that particular bug, and as a novelty, if the mutated code overlaps with the code patch required to fix the bug.

6 CONCLUSION

This paper presents a model-driven approach to mutation testing of Java bytecode, along with the tool MMT. For flexible development of mutation operators, a bytecode model is used that abstracts from actual bytecode. Mutations of Java bytecode are executed as model transformations. We implemented MMT as an Eclipse plug-in and specified a set of 89 mutation operators for Java bytecode, which includes not only basic operators such as manipulations of arithmetic operations or relations, but also operators for mutating the class structure, Java-specific properties, and API method calls.

In our evaluation, we compared MMT with other mutation testing tools for Java bytecode that are not model-based, and with Wodel-Test, a mutation testing tool for Java source code that is also model-based. We found that model-based mutation testing tools are promising for the development of advanced mutation operators, while non-model-based tools come along with fixed operator sets. Similar to the non-model-based tools, MMT proved to be usable for mutation analysis on real projects, here a set of selected Defects4j projects.

Our experiments also showed that MMT, Jumble and PIT have different strengths and weaknesses, so it seems to be a good idea to perform mutation testing of the same project with several tools. Since non-model-based mutation testing tools for Java bytecode have been shown to be faster, efficiency improvements for MMT should be considered in the future. Incremental bytecode generation after mutation and a smarter timeout strategy for infinite-loop detection will be obvious improvements to the runtime performance of MMT. We also plan to further compare MMT with other mutation testing tools for Java, such as Major, Javalanche, and LittleDarwin, to see if the trends reported in this paper will persist. In the wider future, we want to realize a mutation testing approach and tool, which offers a flexible and easy specification of new mutation operators that can be used effectively and efficiently for mutation testing of Java bytecode.[6]

References

[1] 2016. Eclipse Modeling Framework website. https://www.eclipse.org/modeling/emf/.

[2] Sven Amann, Sarah Nadi, Hoan A Nguyen, Tien N Nguyen, and Mira Mezini. 2016. MUBench: A benchmark for API-misuse detectors. In *Proceedings of the 13th international conference on mining software repositories*. 464–467.

[3] Sven Amann, Hoan Anh Nguyen, Sarah Nadi, Tien N Nguyen, and Mira Mezini. 2018. A systematic evaluation of static api-misuse detectors. *IEEE Transactions on Software Engineering* 45, 12 (2018), 1170–1188.

[4] Thorsten Arendt, Enrico Biermann, Stefan Jurack, Christian Krause, and Gabriele Taentzer. 2010. Henshin: Advanced Concepts and Tools for In-Place EMF Model Transformations. In *Proc. MODELS*. Springer, 121–135. https://doi.org/10.1007/978-3-642-16145-2_9

[5] BCEL 2022. The BCEL homepage. https://commons.apache.org/proper/commons-bcel/. Accessed: October 10, 2022.

[6] Enrico Biermann, Claudia Ermel, and Gabriele Taentzer. 2012. Formal foundation of consistent EMF model transformations by algebraic graph transformation. *Softw. Syst. Model.* 11, 2 (2012), 227–250. https://doi.org/10.1007/s10270-011-0199-7

[7] Christoph Bockisch, Gabriele Taentzer, Nebras Nassar, and Lukas Wydra. 2020. Java Bytecode Verification with OCL: Why, How and When. *J. Object Technol.* 19, 3 (2020), 3:1–16. https://doi.org/10.5381/jot.2020.19.3.a13

[8] Christoph Bockisch, Gabriele Taentzer, and Daniel Neufeld. 2023. MMT: Mutation Testing of Java Bytecode with Model Transformation. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 35–39.

[9] Eric Bruneton, Romain Lenglet, and Thierry Coupaye. 2002. ASM: a code manipulation tool to implement adaptable systems. *Adaptable and extensible component systems* 30 (2002), 19. http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.117.5769

[10] Thierry Titcheu Chekam, Mike Papadakis, Yves Le Traon, and Mark Harman. 2017. An empirical study on mutation, statement and branch coverage fault revelation that avoids the unreliable clean program assumption. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 597–608.

[11] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. Pit: a practical mutation testing tool for java. In *Proc. of the 25th Int. Symp. on Software Testing and Analysis*. 449–452.

[12] Pablo Gómez-Abajo, Esther Guerra, Juan de Lara, and Mercedes G. Merayo. 2021. Wodel-Test: a model-based framework for language-independent mutation testing. *Softw. Syst. Model.* 20, 3 (2021), 767–793. https://doi.org/10.1007/s10270-020-00827-0

[13] Sean A Irvine, Tin Pavlinic, Leonard Trigg, John G Cleary, Stuart Inglis, and Mark Utting. 2007. Jumble java byte code to measure the effectiveness of unit tests. In *Testing: Academic and industrial conference practice and research techniques-MUTATION (TAICPART-MUTATION 2007)*. IEEE, 169–175.

[14] Sean A Irvine, Tin Pavlinic, Leonard Trigg, John G Cleary, Stuart Inglis, and Mark Utting. 2007. Jumble: Java byte code to measure the effectiveness of unit tests. In *Testing: Academic and Industrial Conference Practice and Research Techniques*. IEEE, 169–175.

[15] René Just. 2014. The Major mutation framework: Efficient and scalable mutation analysis for Java. In *Proc. of the 2014 Int. Symp. on Software Testing and Analysis*. 433–436.

[16] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.

[17] René Just, Darioush Jalali, Laura Inozemtseva, Michael D Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing?. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 654–665.

[18] Marinos Kintis, Mike Papadakis, Andreas Papadopoulos, Evangelos Valvis, Nicos Malevris, and Yves Le Traon. 2018. How effective are mutation testing tools? An empirical analysis of Java mutation testing tools with manual analysis and real faults. *Empirical Software Engineering* 23 (2018), 2426–2463.

[19] Thomas Laurent, Mike Papadakis, Marinos Kintis, Christopher Henard, Yves Le Traon, and Anthony Ventresque. 2017. Assessing and improving the mutation testing practice of pit. In *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 430–435.

[6]Funding to continue this research has recently been approved by the German Research Foundation (Deutsche Forschungsgemeinschaft) – 529917994.

[20] Jean-Marie Mottu, Benoit Baudry, and Yves Le Traon. 2006. Mutation analysis testing for model transformations. In *Europ. Conf. on Model Driven Architecture-Foundations and Applications*. Springer, 376–390.

[21] Nebras Nassar, Jens Kosiol, Thorsten Arendt, and Gabriele Taentzer. 2018. OCL2AC: Automatic Translation of OCL Constraints to Graph Constraints and Application Conditions for Transformation Rules. In *Graph Transformation - 11th International Conference, ICGT 2018, Proceedings (LNCS, Vol. 10887)*. Springer, 171–177. https://doi.org/10.1007/978-3-319-92991-0_11

[22] Nathaniel Nystrom, Michael R. Clarkson, and Andrew C. Myers. 2003. Polyglot: An Extensible Compiler Framework for Java. In *Compiler Construction*. Springer, 138–152. https://doi.org/10.1007/3-540-36579-6_11

[23] OCL 2014. Object Constraint Language, Version 2.4, Object Management Group. http://www.omg.org/spec/OCL/2.4.

[24] A Jefferson Offutt and Roland H Untch. 2001. Mutation 2000: Uniting the orthogonal. *Mutation testing for the new century* (2001), 34–44.

[25] Jeff Offutt, Yu-Seung Ma, and Yong-Rae Kwon. 2006. The Class-Level Mutants of MuJava. In *Proceedings of the 2006 International Workshop on Automation of Software Test* (Shanghai, China) *(AST '06)*. ACM, 78–84. https://doi.org/10.1145/1138929.1138945

[26] Mike Papadakis, Yue Jia, Mark Harman, and Yves Le Traon. 2015. Trivial compiler equivalence: A large scale empirical study of a simple, fast and effective equivalent mutant detection technique. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 936–946.

[27] Ali Parsai, Alessandro Murgia, and Serge Demeyer. 2017. LittleDarwin: a feature-rich and extensible mutation testing framework for large and complex Java systems. In *Int. Conf. on Fundamentals of Software Engineering*. Springer, 148–163.

[28] David Schuler and Andreas Zeller. 2009. Javalanche: Efficient mutation testing for Java. In *Proc. of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering*. 297–298.

[29] August Shi, Jonathan Bell, and Darko Marinov. 2019. Mitigating the effects of flaky tests on mutation testing. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 112–122.

[30] Bugra M. Yildiz, Christoph Bockisch, Arend Rensink, and Mehmet Aksit. 2017. An MDE Approach for Modular Program Analyses. In *Companion to the 1st Int. Conf. on the Art, Science and Engineering of Programming*, Jennifer B. Sartor, Theo D'Hondt, and Wolfgang De Meuter (Eds.). ACM, 15:1–15:5. https://doi.org/10.1145/3079368.3079392